

PI IV - DESENVOLVIMENTO PARA DISPOSITIVOS MÓVEIS

Prof. Me. Wilson Lourenço

React Native: Componentes Básicos e Projeto Prático com Expo



Exemplo prático de View

Código Simples

```
<View style={{
  padding: 20,
  margin: 10,
  backgroundColor: '#4967E9',
  borderRadius: 12,
  shadowOpacity: 0.2}}>
  <Text>Meu Card</Text>
</View>
```

Propriedades Principais

- **padding:** Espaçamento interno do contêiner
- **margin:** Espaçamento externo em relação a outros elementos
- **backgroundColor:** Cor de fundo do componente
- **borderRadius:** Cantos arredondados elegantes
- **shadow:** Sombras para profundidade visual

Exemplo prático de Text

Tamanhos Variados

```
<Text style={{
  fontSize: 24,
  fontWeight: 'bold'}}>
  Título Grande
</Text>
```

Cores e Alinhamento

```
<Text style={{
  color: '#4967E9',
  textAlign: 'center'}}>
  Texto Centralizado
</Text>
```

Limitação de Linhas

```
<Text numberOfLines={2}>
  Texto longo que
  será cortado em 2
  linhas...
</Text>
```

- A propriedade **numberOfLines** é perfeita para evitar que textos longos quebrem o layout, adicionando reticências automaticamente quando o conteúdo excede o limite definido.

«Image»: Imagens em React Native

→ Fontes Múltiplas

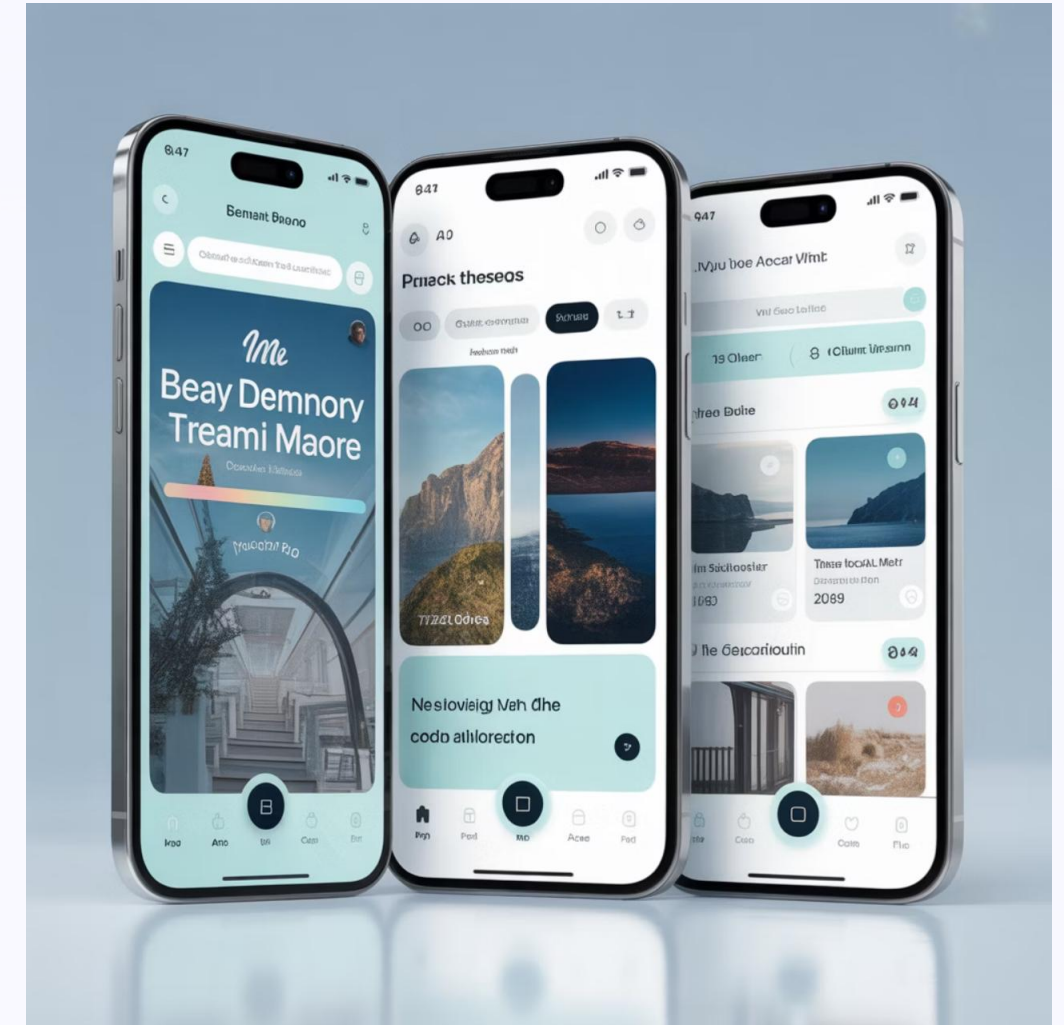
- Suporta tanto imagens locais importadas diretamente no projeto quanto imagens remotas carregadas via URL da internet

→ Interfaces Visuais

- Elemento crucial para criar experiências visuais ricas, atrativas e profissionais que engajam os usuários

→ Controle Total

- Propriedades avançadas permitem ajustar tamanho, modo de redimensionamento (cover, contain, stretch) e proporções



Exemplo prático de Image

Imagem Local

1

```
<Image source={require('./logo.png')}  
        style={{  
            width: 100,  
            height: 100,  
            borderRadius: 50}}  
/>
```

Importe assets locais usando `require()` para garantir otimização durante o build

Imagem Remota

2

```
<Image source={{uri: 'https://...'}}  
        style={{  
            width: '100%',  
            height: 200,  
            resizeMode: 'cover' }}  
/>
```

Use URIs para imagens dinâmicas e `resizeMode` para controlar o comportamento de redimensionamento

«ScrollView»:

Container Inteligente

- Permite rolagem vertical ou horizontal quando o conteúdo excede os limites visíveis da tela



Versatilidade

- Ideal para listas curtas ou qualquer conteúdo que necessite navegação por rolagem

Composição Livre

- Pode conter múltiplos componentes filhos de diferentes tipos simultaneamente



Exemplo prático de ScrollView

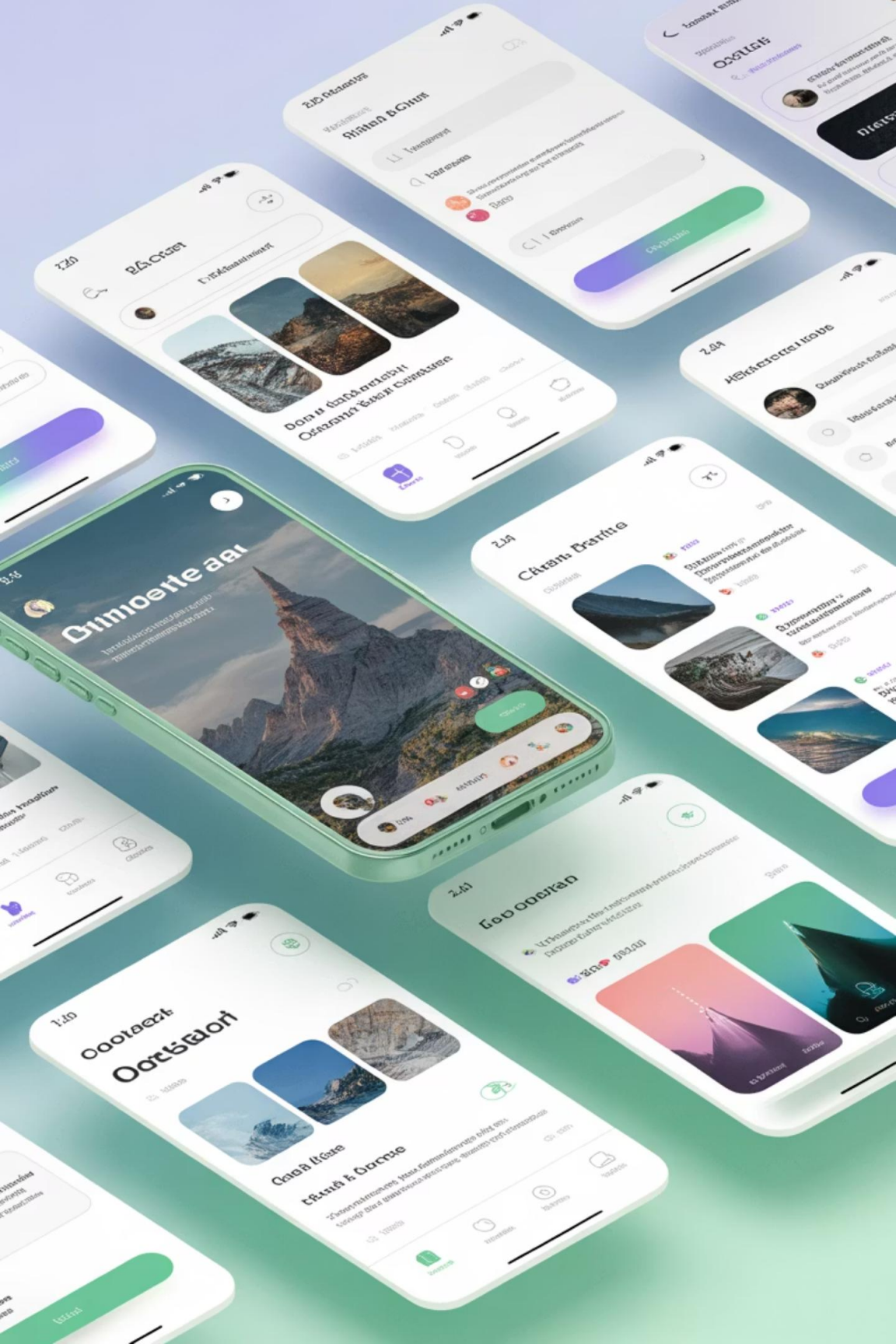
Scroll Vertical

```
<ScrollView>
  <Text>Item 1</Text>
  <Image source={...} />
  <Text>Item 2</Text>
  <Image source={...} />
  <Text>Item 3</Text>
</ScrollView>
```

Scroll Horizontal

```
<ScrollView
  horizontal={true}
  showsHorizontalScrollIndicator={false}>
  <View style={{width: 200}}>
    <Image source={...} />
  </View>
  {/* mais itens */}
</ScrollView>
```

- Configuração padrão permite rolagem vertical natural e intuitiva
- Perfeito para galerias de imagens e carrosséis de conteúdo

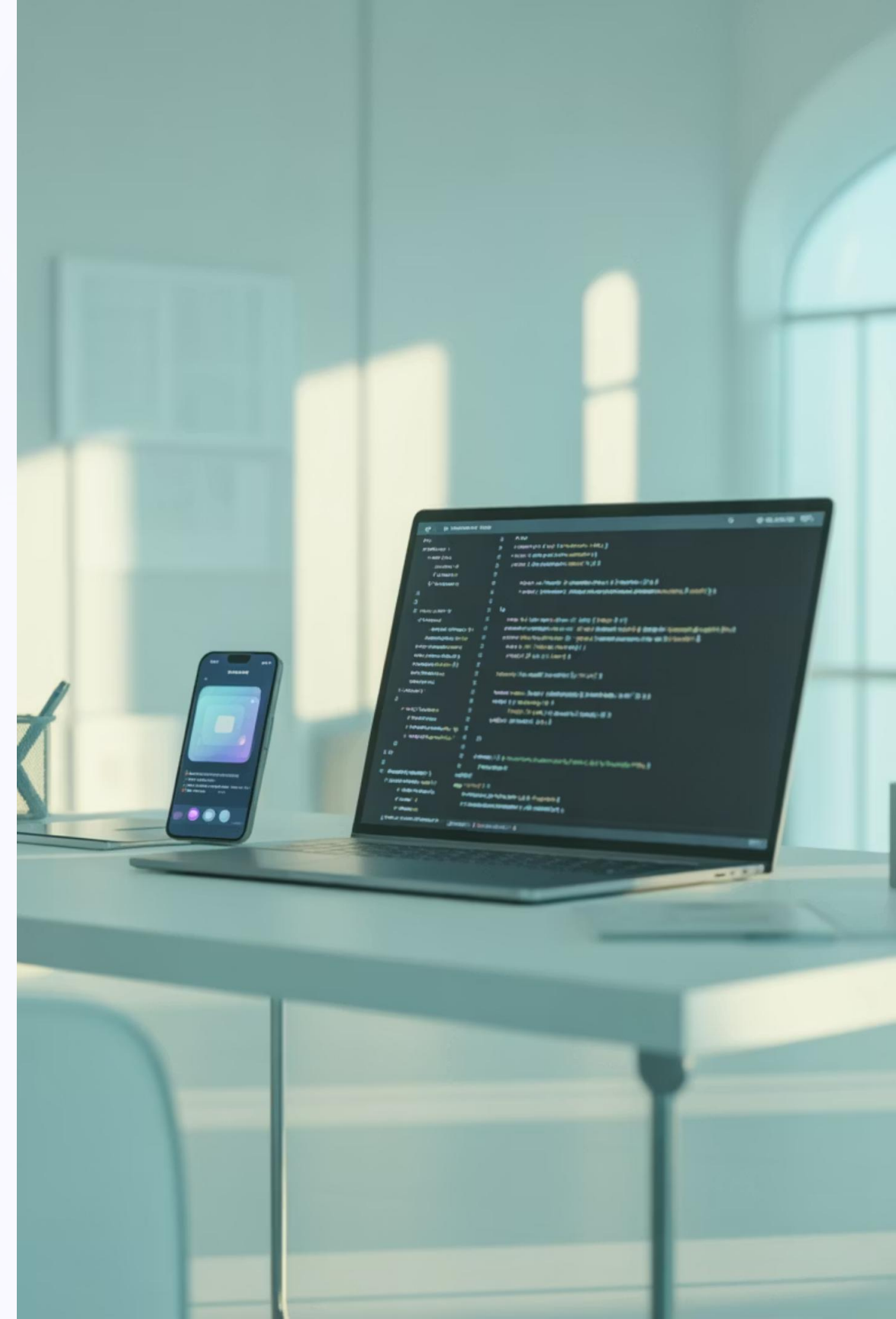


Rolagem suave e flexível

- O ScrollView oferece experiência de navegação fluida e natural, permitindo que usuários explorem conteúdo extenso com gestos intuitivos.
- Perfeito para feeds, galerias e listas dinâmicas que mantêm o engajamento alto.

Projeto Prático no Expo

Criando um App Simples



Passo 1: Criando o projetoprjMeuAppJS

Abra o terminal e digite:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

```
npx create-expo-app prjMeuAppJS --template blank@sdk-54
```

```
npx expo start -c
```

Estrutura Básica

O arquivo **App.js** será criado como ponto de entrada principal, já com importações essenciais de View, Text, Image e ScrollView

Pronto para Desenvolver

Em poucos segundos, você terá um ambiente completo e funcional para começar a construir seu aplicativo imediatamente



Passo 2: Rodando e testando no Expo Go



```
PROBLEMAS  SAÍDA  TERMINAL  PORTAS  GITLENS  IBM I JOB LOG  CONSOLE DO Z/OS
>  ▾ TERMINAL
PS D:\Projetos\VSCode\React\meu-app-js> npx expo start
Starting project at D:\Projetos\VSCode\React\meu-app-js
Starting Metro Bundler



> Metro waiting on exp://10.35.28.177:8081
> Scan the QR code above with Expo Go (Android) or the Camera app (iOS)

> Web is waiting on http://localhost:8081

> Using Expo Go
> Press s | switch to development build

> Press a | open Android
> Press w | open web

> Press j | open debugger
> Press r | reload app
> Press m | toggle menu
> shift+m | more tools
> Press o | open project code in your editor

> Press ? | show all commands

Logs for your project will appear below. Press Ctrl+C to exit.
```



Teste Instantâneo

- Digite `npx expo start`;
- Escaneie o QR code com Expo Go no seu celular e;
- Veja o app funcionando em segundos, sem compilação complexa

Hot Reload



Hot Reload Mágico

- Altere o Código (próximo slide) e veja as mudanças aparecerem no dispositivo em tempo real, acelerando o desenvolvimento



Ajustes Responsivos

- Teste em diferentes dispositivos e tamanhos de tela, ajustando estilos para garantir experiência perfeita em todos

Altere o código para ver o hot reload!

```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>
        Hello React Native!
      </Text>
      <StatusBar style="auto" />
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#4967E9',
  },
});
```



Estilização com StyleSheet no React Native

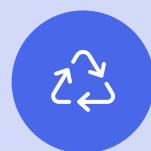
Estilização e desenvolvimento prático com Expo Go

Por que estilizar com StyleSheet?



Organização

- Separa completamente os estilos da lógica de negócio, tornando o código mais limpo, legível e facilitando a colaboração em equipe



Reaproveitamento

- Permite criar estilos reutilizáveis em múltiplos componentes, reduzindo redundância e facilitando manutenções futuras



Validação

- Oferece verificação estática de tipos e detecta erros de sintaxe antes da execução, prevenindo bugs comuns de estilização

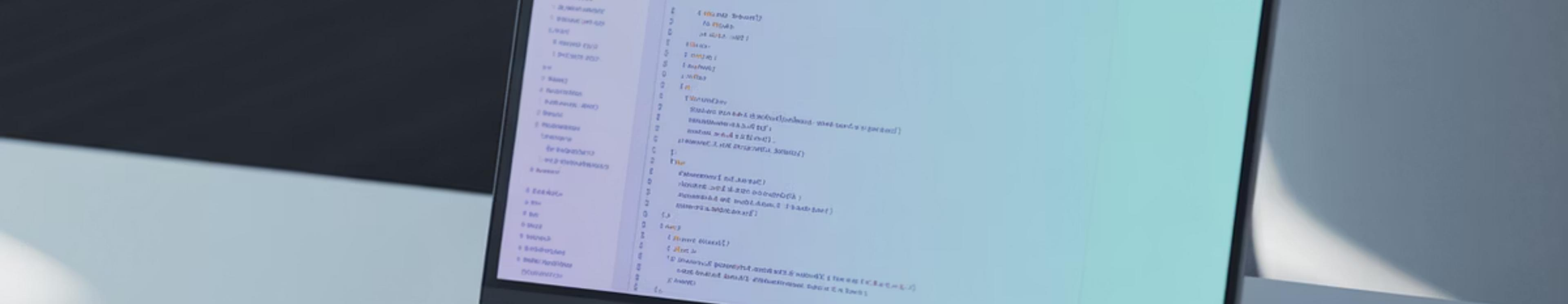
O que é StyleSheet?

Abstração poderosa

- StyleSheet é similar ao CSS, mas otimizada especificamente para React Native.
- Funciona de forma consistente em iOS e Android.
- Criado através do método `StyleSheet.create()`, que:
 - Valida propriedades e;
 - Otimiza o processamento para melhor performance.

Benefícios principais

- Validação automática de propriedades
- Otimização de performance em tempo de execução
- Garantia de consistência multiplataforma



Estilos organizados e performáticos

O método `StyleSheet.create()` é a base para criar estilos eficientes, validados e otimizados automaticamente pelo React Native.

Métodos principais do StyleSheet

01

`create()`

- Cria um estilo com validação automática de propriedades e otimização de performance

02

`compose()`

- Combina dois estilos, onde o segundo sobrescreve propriedades conflitantes do primeiro

03

`flatten()`

- Mescla múltiplos estilos em um único, útil para composição dinâmica e condicional

Propriedades úteis do StyleSheet

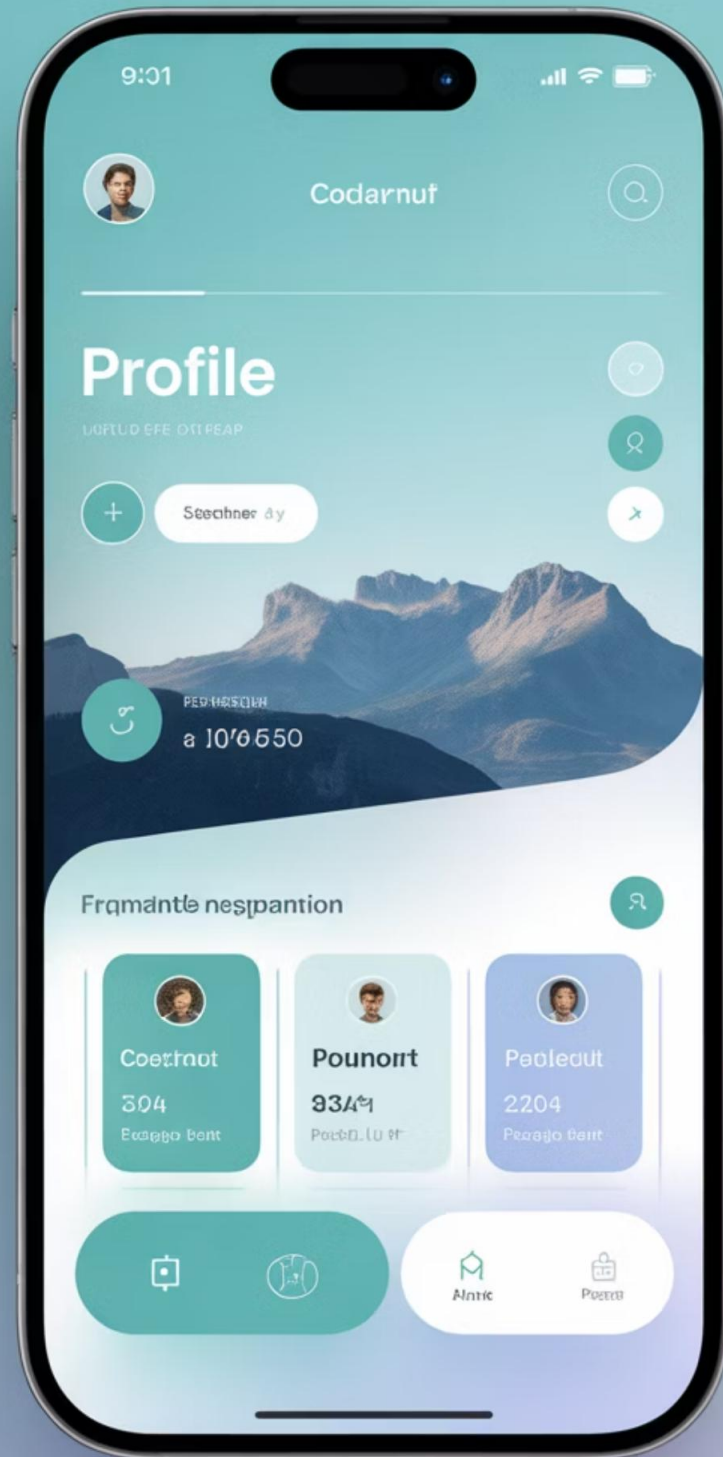
absoluteFill

- Posicionamento absoluto que preenche completamente o container pai, ideal elementos de fundo que cobrem toda a tela

hairlineWidth

- Define a menor largura possível para bordas nativas, adaptando-se automaticamente à densidade de pixels de diferentes dispositivos para bordas sempre finas e nítidas





Projeto prático:

Cartão de perfil estilizado

- Vamos criar um componente de cartão de perfil completo, aplicando os conceitos de StyleSheet para construir uma interface elegante e responsiva.

Criando o projeto

```
#Volte para a pasta raiz dos projetos
```

```
cd..
```

```
#Crie um novo projeto
```

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

```
npx create-expo-app prjPerfilCard --template blank@sdk-54
```

- Apague tudo e digite o código dos próximo 3 slides:

Código fonte:

```
JS App.js M X
prjPerfilCard > JS App.js > ...
You, há 4 dias | 1 author (You)
1 import { StatusBar } from 'expo-status-bar';
2 import React from 'react'; 7.9k (gzipped: 3.1k)
3 import { View, Text, Image } from 'react-native';
4 import styles from './styles';
5
Explain Code | Generate Tests | Generate Docstrings | Ask Sourcery
6 const ProfileCard = () =>{
7   return(
8     <View style={styles.card}>
9       <Image source={{uri: 'https://picsum.photos/100'}}
10        style={styles.avatar} />
11       <Text style={styles.name}>Maria Silva</Text>
12       <Text style={styles.role}>Desenvolvedora React Native</Text>
13       <View style={styles.stats}>
14         <View style={styles.statItem}>
15           <Text style={styles.statValue}>127</Text>
16           <Text style={styles.statLabel}>Projetos</Text>
17         </View>
18         <View style={styles.statItem}>
19           <Text style={styles.statValue}>2.5k</Text>
20           <Text style={styles.statLabel}>Seguidores</Text>
21         </View>
22       </View>
23     </View>
24   )
25 }
```

Estilização dos components:

```
prjPerfilCard > JS styles.js > ...
1 import { StyleSheet } from 'react-native';
2
3 const styles = StyleSheet.create({
4   card: {
5     backgroundColor: '#fff',
6     borderRadius: 16,
7     padding: 24,
8     alignItems: 'center',
9     shadowColor: '#000',
10    shadowOffset: {
11      width: 0,
12      height: 2,
13    },
14    shadowOpacity: 0.1,
15    shadowRadius: 8,
16    elevation: 4,
17  },
18   avatar: {
19     width: 100,
20     height: 100,
21     borderRadius: 50,
22     marginBottom: 16,
23     borderWidth: 3,
24     borderColor: '#4967E9',
25   },
26   name: {
27     fontSize: 24,
28     fontWeight: 'bold',
29     color: '#333',
30     marginBottom: 4,
31   },
32 });
```

Estilização dos componentes:

```
32     role: {
33       fontSize: 16,
34       color: '#666',
35       marginBottom: 20,
36     },
37     stats: {
38       flexDirection: 'row',
39       justifyContent: 'space-between',
40       width: '100%',
41       paddingTop: 20,
42       borderTopWidth: StyleSheet.hairlineWidth,
43       borderTopColor: '#e0e0e0',
44     },
45     statItem: {
46       alignItems: 'center',
47     },
48     statValue: {
49       fontSize: 20,
50       fontWeight: 'bold',
51       color: '#4967E9',
52     },
53     statLabel: {
54       fontSize: 14,
55       color: '#999',
56     },
57   });
58
59   export default styles;
```

Criando o projeto

```
#Navegar para o Diretório
```

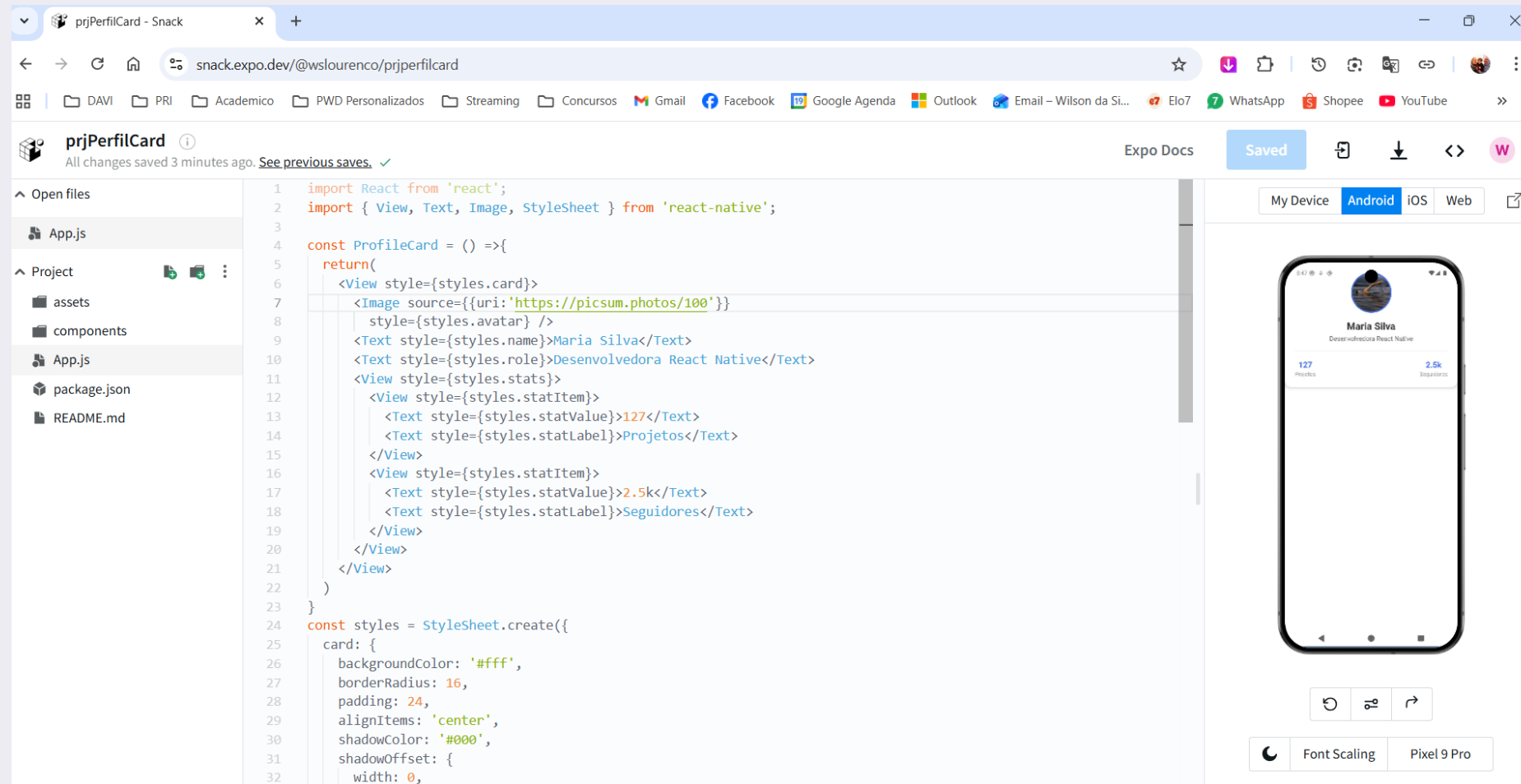
```
cd prjPerfilCard
```

```
#Iniciar o servidor de Desenvolvimento
```

```
npx expo start
```

- Após iniciar, um QR Code aparecerá no terminal.
- Escaneie com o app Expo Go para visualizar seu aplicativo no dispositivo ou;
- Vá para o próximo slide e Use o Expo Snacks!

Expo Snacks - snack.expo.dev



- Você também pode rodar os seus projetos no Expo Snacks



React Native: Componentes Básicos

O que são Componentes no React Native?

Componentes são os **blocos de construção fundamentais** que definem tanto a aparência quanto o comportamento da interface do seu aplicativo.

Componentes Básicos

- **<View>** - Container para layout
- **<Text>** - Exibição de texto
- **<Image>** - Imagens e ícones
- **<ScrollView>** - Barra de rolagem

Componentes podem ser **aninhados** uns dentro dos outros para criar interfaces complexas de forma modular e organizada.



Hierarquia de Componentes



View

Componente container principal

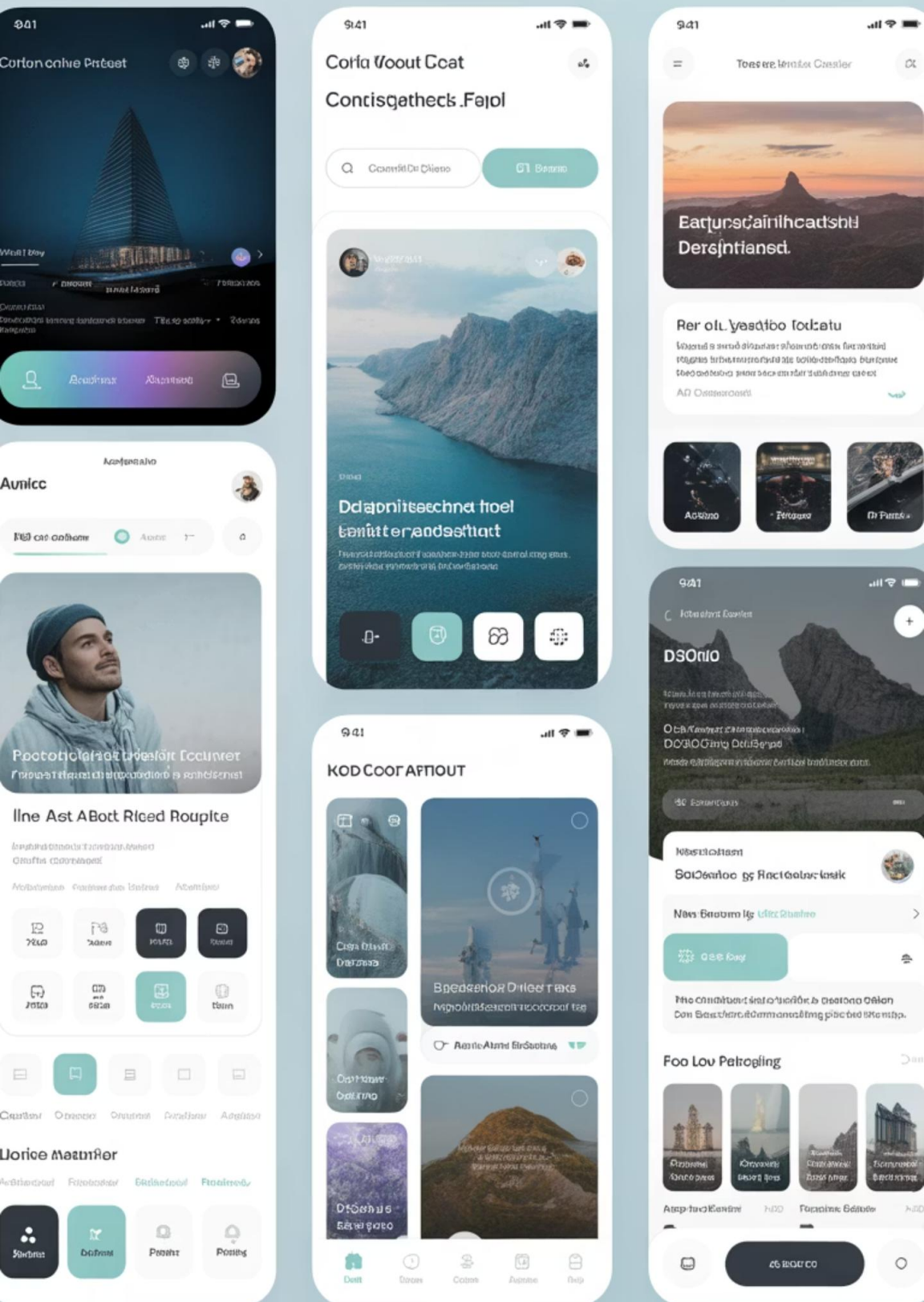
Text

Componente filho para texto

Image

Componente filho para imagens

- Cada componente pode conter outros componentes, criando uma árvore hierárquica que define toda a estrutura visual do aplicativo.



Componente <View>

Container Fundamental

- O <View> é o contêiner básico para agrupamento e organização de elementos na tela.
- Essencial para criar qualquer layout.

Equivalente Nativo

- Corresponde ao UIView no iOS e ViewGroup no Android, garantindo renderização nativa e performance otimizada em ambas plataformas.

Flexbox Integrado

- Suporta Flexbox nativamente, permitindo criar layouts responsivos que se adaptam a diferentes tamanhos de tela com facilidade.

Componente <Text>

Renderização de Texto

- O componente <Text> é responsável por **exibir qualquer conteúdo textual** no aplicativo, desde títulos até parágrafos longos.

Recursos Principais

- Estilização rica com fontes e cores
- Suporte a eventos de toque
- Aninhamento de componentes Text
- Formatação inline flexível

É a base para **toda comunicação textual** com o usuário.



Componente <Image>



Avatar de Perfil

Ideal para fotos de usuário com bordas arredondadas



Imagens de Produtos

Exiba catálogos e galerias com qualidade



Ícones e Logos

Suporte a SVG e diferentes formatos

- O <Image> suporta **imagens locais e remotas**, diferentes formatos (PNG, JPG, SVG) e oferece controles avançados de redimensionamento e estilização.

Componente <ScrollView>

01

Container de rolagem

Permite que o conteúdo ultrapasse os limites da tela, habilitando rolagem vertical ou horizontal.

03

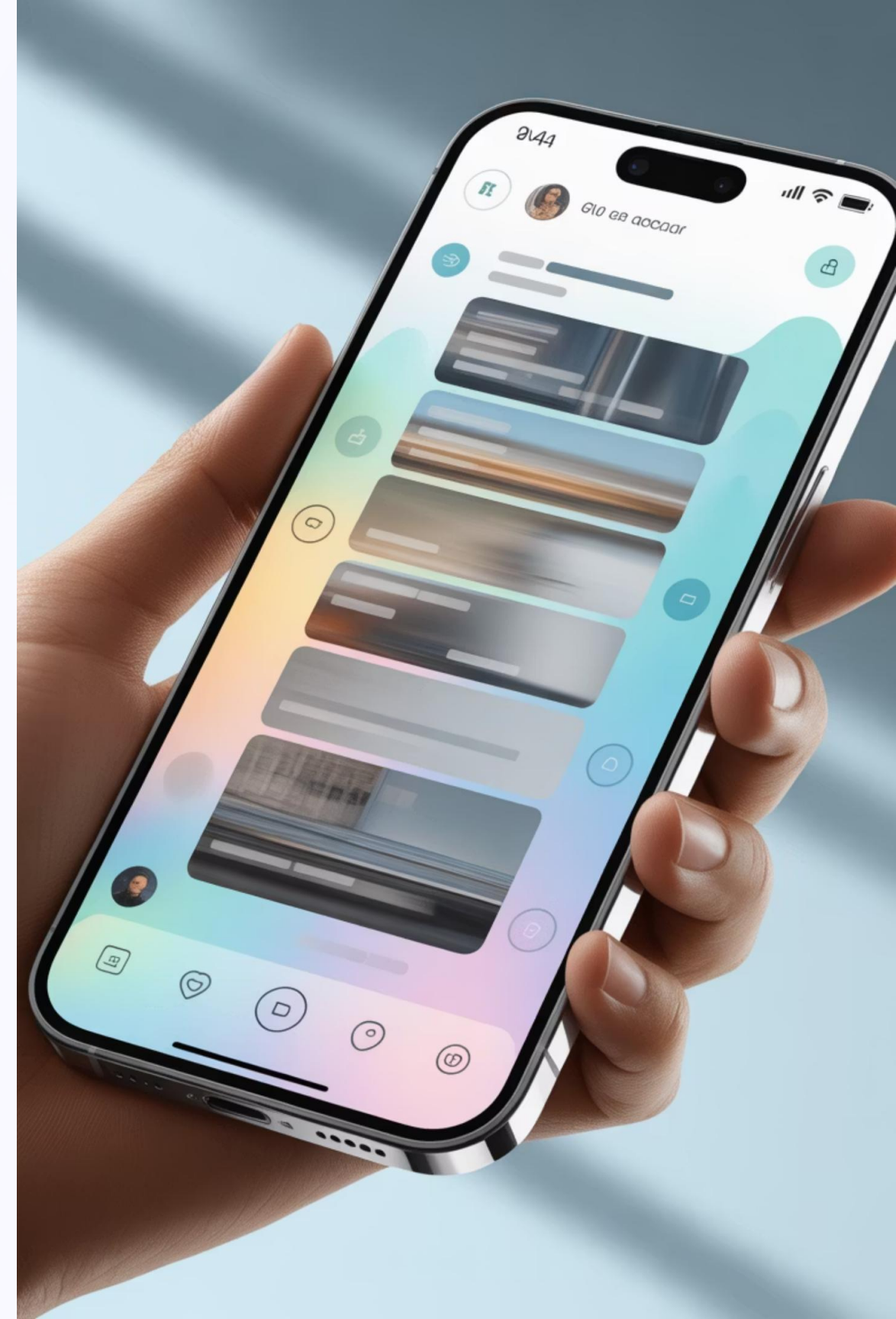
Múltiplos Componentes

Pode conter qualquer número e tipo de componentes filhos organizados sequencialmente.

02

Ideal para Listas Curtas

Perfeito para conteúdo que não cabe na tela, como formulários longos ou artigos extensos.



Componente <TextInput>



Entrada de Dados do Usuário

- O <TextInput> é o componente essencial para **capturar informações** digitadas pelo usuário.

Funcionalidades

- Teclado nativo otimizado
- Placeholders informativos
- Validação em tempo real
- Eventos onChange e onSubmit
- Máscaras de formatação

Fundamental para formulários, telas de login, cadastro e edição de perfil.

Estrutura Básica dos Projetos

Arquitetura do Aplicativo

- Todo projeto React Native possui uma **estrutura organizada** que facilita o desenvolvimento e manutenção.

Elementos Principais

- **App.js** - Ponto de entrada
- **package.json** - Dependências
- **assets/** - Imagens e recursos
- **components/** - Componentes customizados

❏ **Dica:** Organize seus componentes em pastas separadas para projetos maiores, facilitando a navegação e reutilização de código.



Exercício Prático: Tela de Perfil

1

Objetivo do Exercício

- Criar uma tela completa de perfil de usuário com foto, informações pessoais e botão de ação.

2

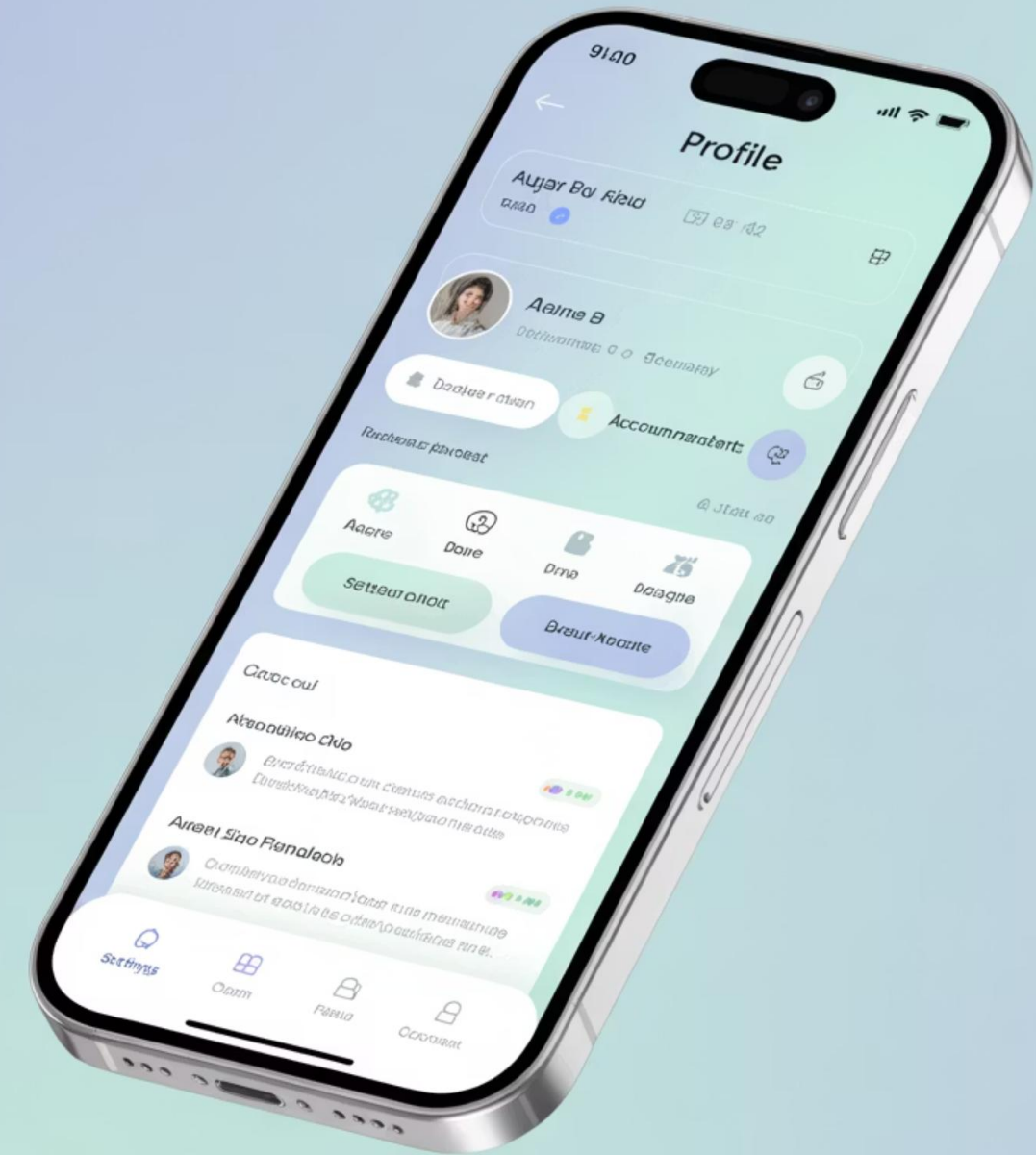
Componentes Utilizados

- **View** (container), **Image** (foto), **Text** (nome e descrição), **Pressable** (botão interativo).

3

Aprendizado Prático

- Combine estilos, layout flexbox e eventos de toque para criar uma interface funcional e profissional.



Passo 1: Container Principal

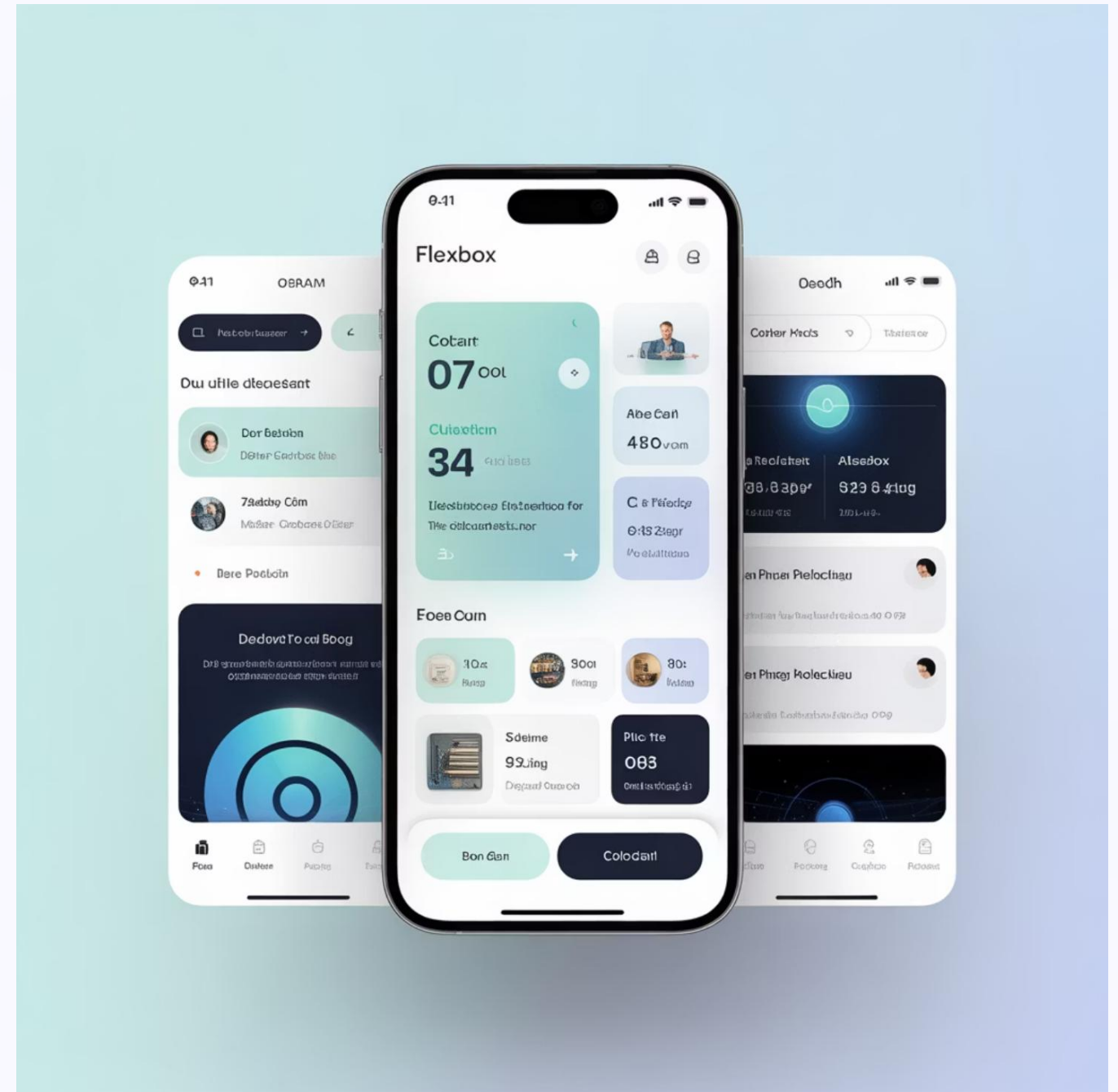
Criar o <View> Principal

O primeiro passo é definir um container que **centraliza todo o conteúdo** da tela usando Flexbox.

Propriedades de Estilo

- `flex: 1` - Ocupa toda tela
- `justifyContent: 'center'` - Centraliza verticalmente
- `alignItems: 'center'` - Centraliza horizontalmente
- `backgroundColor: '#f5f5f5'` - Cor de fundo

Esse layout garante que todos os elementos fiquem **perfeitamente centralizados** na tela.



Passo 2: Foto do Perfil



Estilo Circular

Use `borderRadius: 60` com dimensões `120x120` para criar um avatar circular perfeito.

Propriedades do Componente <Image>

```
<Image source={{uri: 'URL_DA_FOTO'}}  
  style={{  
    width: 120,  
    height: 120,  
    borderRadius: 60,  
    marginBottom: 16  
  }}  
</>
```

- Você pode usar uma **URL remota** ou uma imagem local da pasta `assets/` do projeto.



Passo 3: Nome e Descrição

Nome do Usuário

- Use `<Text>` com `fontSize: 24,` `fontWeight: 'bold'` e cor escura para destacar o nome principal.

Descrição ou Bio

- Texto menor com `fontSize: 16,`
- `color: '#666'` e `textAlign: 'center'` para informações secundárias.

Espaçamento

Use `marginTop` e `marginBottom` entre elementos para criar hierarquia visual clara e respiração.

Passo 4: Botão Interativo

Componente <Pressable>

- O Pressable oferece **feedback visual** ao toque e permite customização completa do comportamento do botão.

Implementação

- Background color para destaque
- Padding interno para área clicável
- BorderRadius para cantos arredondados
- Evento `onPress` para ação
- Mudança de opacidade ao pressionar

❏ **Exemplo de Ação:** Use `Alert.alert('Sucesso!', 'Perfil salvo com sucesso!')` para feedback imediato ao usuário.



Criando o projeto

```
#Volte para a pasta raiz dos projetos
```

```
cd..
```

```
#Crie um novo projeto
```

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

```
npx create-expo-app prjPerfilUsuario --template blank
```

- Apague tudo e digite o código dos próximo 3 slides:

Código Completo do Exemplo

```
JS App.js M X
prjPerfilUsuario > JS App.js > App
You, há 5 minutos | 1 author (You)
1 import { StatusBar } from 'expo-status-bar';
2 import { View, Text, Image, Pressable, Alert } from 'react-native';
3 import styles from './styles';
4
Explain Code | Generate Tests | Generate Docstrings | Ask Sourcery
5 export default function App() {
6   return (
7     <View style={styles.container}>
8       <Image
9         source={{uri: 'https://picsum.photos/100'}}
10        style={styles.avatar}
11      />
12      <Text style={styles.name}>
13        João Silva
14      </Text>
15      <Text style={styles.description}>
16        Desenvolvedor React Native apaixonado por criar experiências mobile incríveis
17      </Text>
18      <Pressable
19        onPress={() => Alert.alert('Sucesso!', 'Perfil salvo!')}
20        style={styles.button}
21      >
22        <Text style={styles.buttonText}>
23          Editar Perfil
24        </Text>
25      </Pressable>
26    </View>
27  );
28 }
```

Código Completo do Exemplo

```
JS styles.js U X
prjPerfilUsuario > JS styles.js > ...
1  import { StyleSheet } from 'react-native';
2
3  export default StyleSheet.create({
4    container: {
5      flex: 1,
6      justifyContent: 'center',
7      alignItems: 'center',
8      backgroundColor: '#f5f5f5',
9    },
10   avatar: {
11     width: 120,
12     height: 120,
13     borderRadius: 60,
14     marginBottom: 16,
15   },
16   name: {
17     fontSize: 24,
18     fontWeight: 'bold',
19     marginBottom: 8,
20   },
```

Código Completo do Exemplo

```
21   description: {
22     fontSize: 16,
23     color: '#666',
24     textAlign: 'center',
25     marginBottom: 24,
26     paddingHorizontal: 32,
27   },
28   button: {
29     backgroundColor: '■ #4967E9',
30     paddingVertical: 12,
31     paddingHorizontal: 32,
32     borderRadius: 8,
33   },
34   buttonText: {
35     color: 'white',
36     fontSize: 16,
37     fontWeight: '600',
38   },
39 };
```

Expo Snacks

prjPerfilUsuario - Snack

snack.expo.dev

DAVI PRI Academico PWD Personalizados Streaming Concursos Gmail Facebook Google Agenda Outlook Email – Wilson da Si... Elo7 WhatsApp Shopee YouTube

prjPerfilUsuario

All changes saved 4 minutes ago. [See previous saves.](#)

Expo Docs Save

My Device Android iOS Web

Download [Expo Go](#) and scan the QR code to get started.



Open files

App.js styles.js

Project

assets components App.js package.json README.md styles.js

```
1 import { StyleSheet } from 'react-native';
2
3 export default StyleSheet.create({
4   container: {
5     flex: 1,
6     justifyContent: 'center',
7     alignItems: 'center',
8     backgroundColor: '#f5f5f5',
9   },
10  avatar: {
11    width: 120,
12    height: 120,
13    borderRadius: 60,
14    marginBottom: 16,
15  },
16  name: {
17    fontSize: 24,
18    fontWeight: 'bold',
19    marginBottom: 8,
20  },
21  description: {
22    fontSize: 16,
23    color: '#666',
24  },
25  ...
--INSERT--
```

No errors

Prettier {} Editor Expo v53.0.0 Devices 1 Preview

✓ Agora você tem uma tela de perfil funcional! Execute com `npx expo start` ou o Expo Snacks

Dicas para Estudo



Seja “CURIOSO”:

Procure revisar o que foi estudado.
Pesquise as referências bibliográficas.



Seja “ANTENADO”:

Leia a próxima aula.



Seja “COLABORATIVO”:

Traga assuntos relevantes para a sala de aula.
Participe da aula.
Proponha discussões relevantes sobre o conteúdo.



Prof. Me. Wilson Lourenço

